

WeatherForecast System

Student Name: Melvin Jose

Register Number: AJC23CS140

Class: S6 CSE-C

Department: Computer Science and Engineering

Abstract

The WeatherForecast System is a Django-based web application developed to provide accurate, user-friendly weather updates through both manual city/ZIP search and geolocation-based insights. The system integrates real-time weather APIs, caches responses to reduce latency and API usage, and presents data using a responsive glassmorphism interface. In addition to current conditions and multi-day forecast cards, it includes actionable features such as rain-start prediction, outdoor planning guidance, air-quality tips, and city-to-city comparison. The project emphasizes clean architecture by separating service logic from views and enhancing usability with autocomplete, instant unit switching, theme toggle, and robust error handling for invalid locations and network issues.

Problem Description

- Weather information is often distributed across multiple apps and websites, making it difficult for users to quickly make daily decisions.
- Basic forecast pages may show raw temperature only, without contextual insights such as rain timing, outdoor suitability, and air-quality advisories.
- Frequent API calls can also cause slow performance and rate-limit issues if caching is not implemented.
- This project solves these issues by providing a single, insight-driven interface that combines real-time data, location awareness, and optimized caching.

Libraries & Tools Used (with purpose)

- Django (5.x): Web framework used for URL routing, view rendering, template handling, and project structure management.
- requests: Used in service layer to fetch weather, forecast, geocoding, and air-pollution data from external APIs.
- python-dotenv: Loads API key and configuration values from .env for secure secret management.
- OpenWeatherMap APIs: Primary data provider for current weather, forecast, geolocation lookup, and AQI inputs.
- Django Cache (LocMemCache): Stores API responses temporarily to reduce repeated calls and improve speed.
- Bootstrap + Custom CSS: Provides responsive layout foundation and custom visual design.
- JavaScript (Vanilla): Implements geolocation flow, autocomplete, theme toggling, and dynamic insight interactions.

System Design / Workflow

- User opens the homepage and enters a location or uses location permission.
- Frontend sends query or coordinates to Django endpoint.
- View delegates weather retrieval to services.py.
- Service checks cache; if not available, calls OpenWeather APIs.
- Service parses required fields and computes insight summaries.
- Django renders structured context into templates.
- UI displays current temperature, conditions, forecast, and feature outputs.

Implementation Details

- Project follows modular architecture with weather_project (settings/routing) and forecast app (views/services/templates).
- Services layer isolates API integration, response parsing, temperature conversion, and insight generation logic.
- Search endpoint supports both text query and coordinate-based fetch mode.
- Autocomplete endpoint returns suggestions for user-typed location strings.
- Feature insights endpoint returns domain-specific outputs (rain alert, outdoor planner, AQ tips, city compare).
- Current-summary endpoint supports homepage temperature card and fast location-based rendering.
- Frontend enforces manual-search validation ("Type place") and separates it from "Use my location" flow.
- Theme system supports standard mode and storm-cloud blue mode with persistent preference.
- Client-side and server-side caching are both used to improve perceived performance.

Results & Output

- The system successfully displays real-time weather information with user-friendly visualization and responsive behaviour.
- Manual search and geolocation both work with validation and fallback handling.
- Insight panels generate contextual outputs that are more actionable than raw weather metrics.
- Caching improves repeated-load speed and reduces redundant API requests.

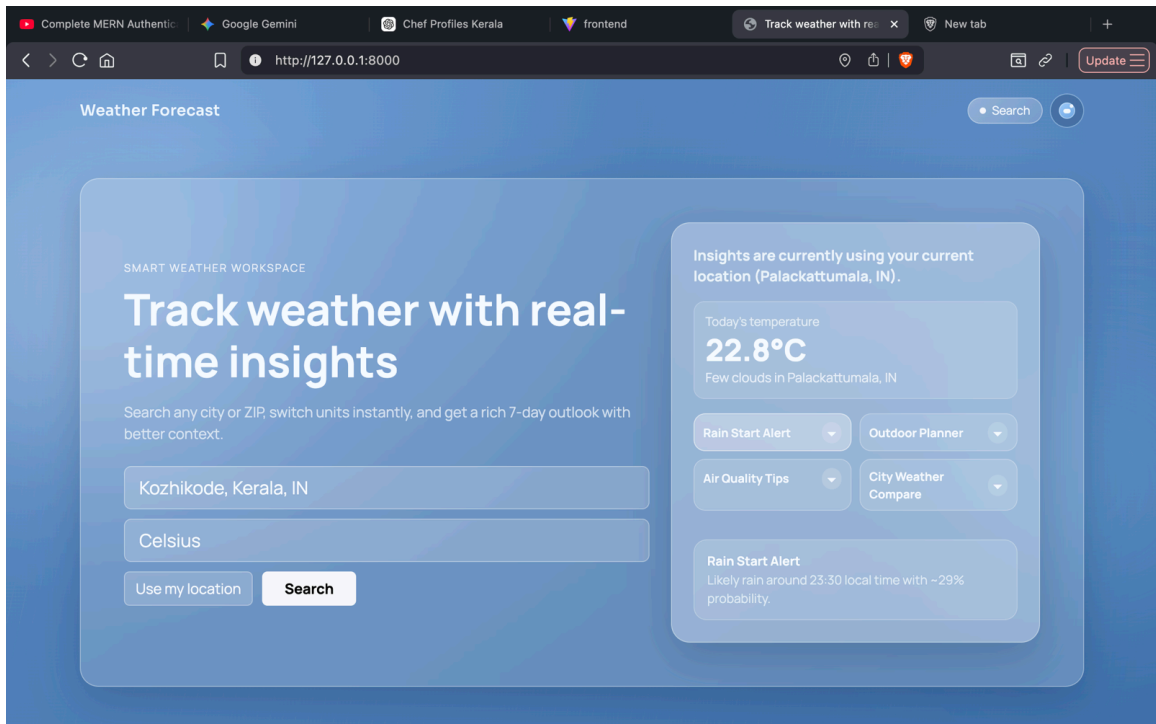


Figure 1: Input / Interface / Start Screen (Home page with search and real-time insights panel).

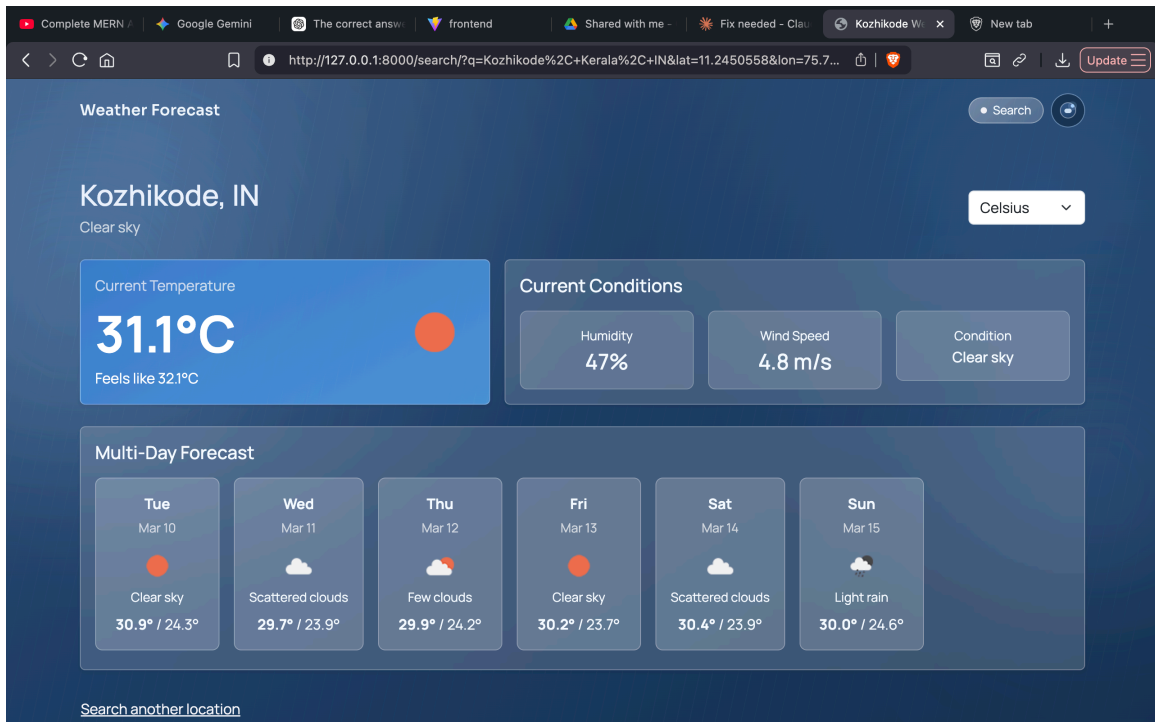


Figure 2: Final Output / Result (WeatherForecast system output after feature execution).

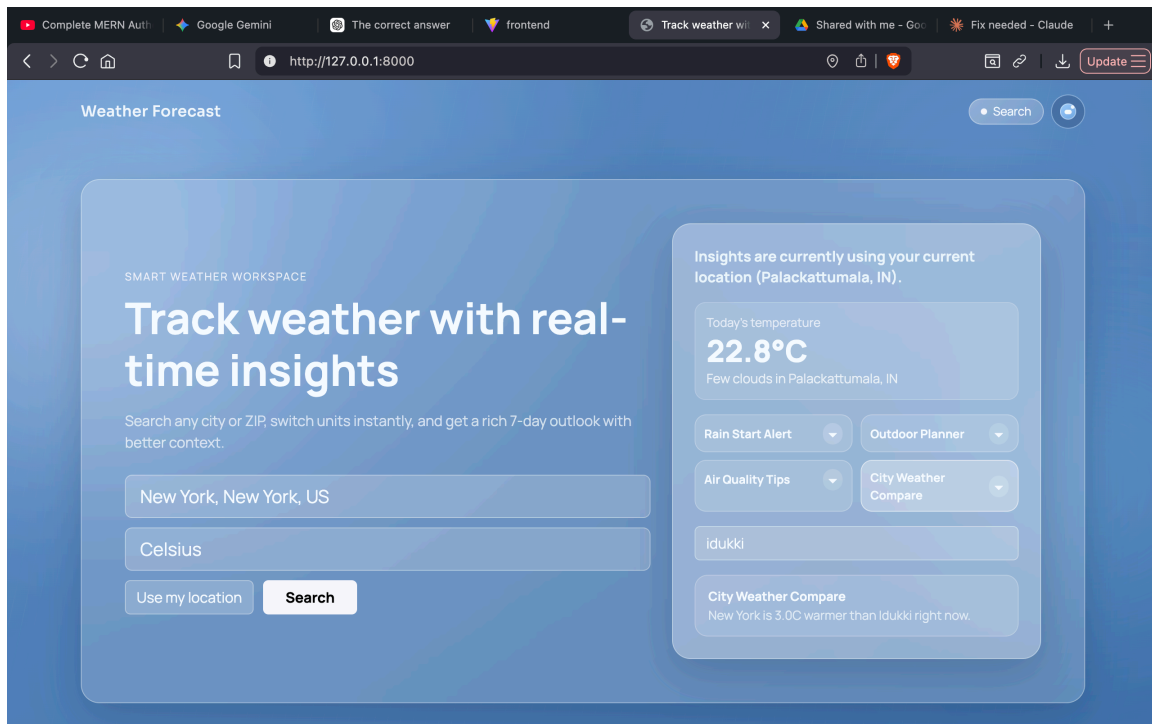


Figure 3: City Weather Comparison Feature

8. Conclusion & Scope for Mini Project Extension

- The WeatherForecast System demonstrates a complete end-to-end mini project covering frontend UI, backend services, API integration, caching, and user-centric features.
- The current version meets core requirements and provides practical weather insights with good usability.
- Scope for extension in mini project phase includes: push notifications for weather alerts, user accounts with favourite cities, map-based radar visualization, multilingual interface, and historical weather analytics dashboards.

9. References

1. Django Documentation: <https://docs.djangoproject.com/>
2. OpenWeatherMap API Documentation: <https://openweathermap.org/api>
3. Requests Documentation: <https://requests.readthedocs.io/>
4. python-dotenv Documentation: <https://pypi.org/project/python-dotenv/>
5. Bootstrap Documentation: <https://getbootstrap.com/docs/>